



CẤU TRÚC DỮ LIỆU & GIẢI THUẬT

BÀI 5: Đồ thị và thuật toán duyệt

PHẦN 1



Đồ thị là gì?

Đồ thị $G = (V, E)$ là một cấu trúc dữ liệu gồm:

V (Vertices): Tập các đỉnh (nodes)

E (Edges): Tập các cạnh kết nối giữa các đỉnh

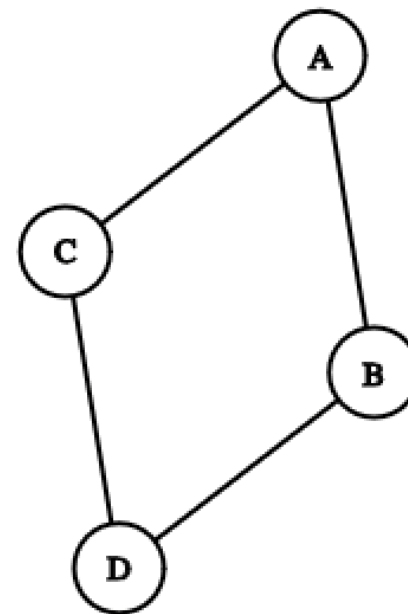
Ví dụ:

$V = \{A, B, C, D\}$

$E = \{(A,B), (A,C), (B,D), (C,D)\}$

Đồ thị xuất hiện ở đâu?

Mạng xã hội, bản đồ đường đi, mạng máy tính, quan hệ dữ liệu...



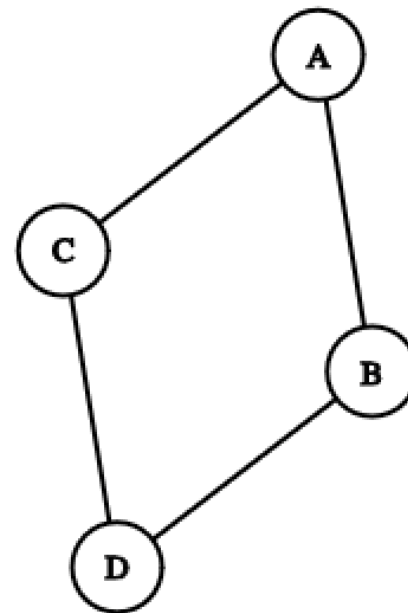
Các khái niệm cơ bản

- Đỉnh (Vertex/Node): Điểm trong đồ thị, đại diện cho một đối tượng
- Cạnh (Edge): Kết nối giữa 2 đỉnh, đại diện cho quan hệ
- Đồ thị có hướng (Directed Graph): Cạnh có chiều: $A \rightarrow B$ (khác $B \rightarrow A$)
- Đồ thị vô hướng (Undirected Graph): Cạnh không có chiều: $A — B$ (A đến B = B đến A)
- Bậc (Degree): Số cạnh nối với đỉnh đó

Đồ thị có hướng vs vô hướng

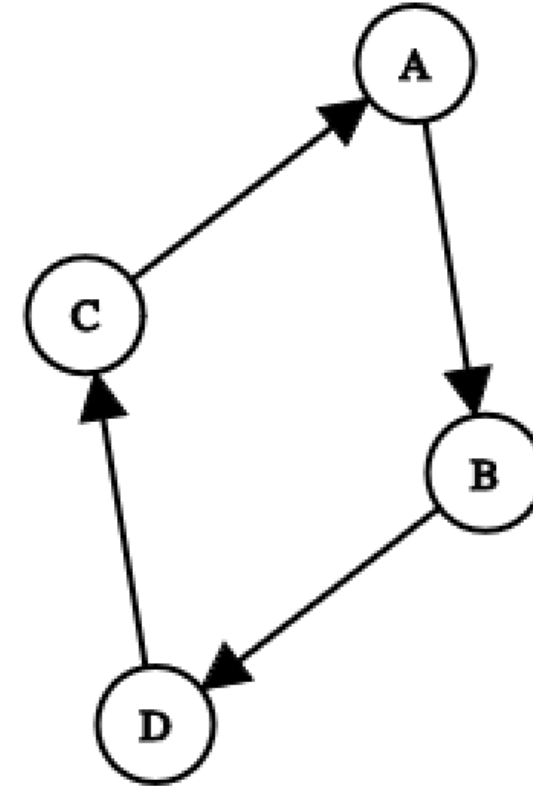
Đồ thị vô hướng (Undirected):

- Cạnh: $\{A-B, A-C, B-D, C-D\}$
- A đến B = B đến A (hai chiều)
- Ví dụ: Mạng bạn bè Facebook



Đồ thị có hướng (Directed):

- Cạnh: $\{A \rightarrow B, B \rightarrow D, D \rightarrow C, C \rightarrow A\}$
- A đến B $\neq$ B đến A (một chiều)
- Ví dụ: Twitter follow, đường một chiều



Đồ thị có trọng số

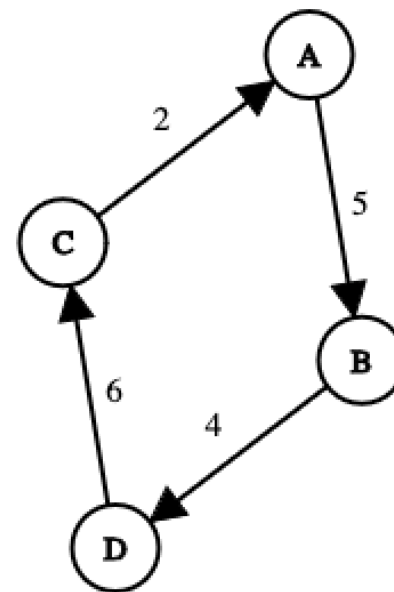
Đồ thị có trọng số (Weighted Graph): Mỗi cạnh có một giá trị (trọng số, weight)

Ví dụ: Bản đồ đường đi

- Cạnh (A,B) có trọng số 5 (khoảng cách 5km)
- Cạnh (A,C) có trọng số 2 (khoảng cách 2km)

Ứng dụng:

Tìm đường đi ngắn nhất, chi phí tối thiểu, thời gian nhanh nhất



Biểu diễn đồ thị trong máy tính

Có 2 cách chính:

- Ma trận kề (Adjacency Matrix): Dùng mảng 2 chiều $n \times n$
- Danh sách kề (Adjacency List): Dùng dictionary hoặc list of lists

Câu hỏi: Nên dùng cách nào?

- Phụ thuộc vào độ dày đặc của đồ thị
- Dense graph (nhiều cạnh) $\rightarrow$ Ma trận kề
- Sparse graph (ít cạnh) $\rightarrow$ Danh sách kề

Ma trận kề (Adjacency Matrix)

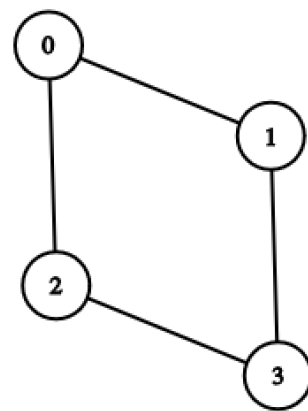
Định nghĩa: Ma trận 2 chiều $n \times n$, với $matrix[i][j] = 1$ nếu có cạnh từ i đến j

Ưu điểm:

- Kiểm tra cạnh (i,j) trong $O(1)$
- Đơn giản, dễ cài đặt

Nhược điểm:

- Tốn $O(n^2)$ không gian (lãng phí với đồ thị thưa)
- Duyệt hàng xóm của đỉnh tốn $O(n)$



Ma trận kề:

	0	1	2	3
0	[0	1	1	0]
1	[1	0	0	1]
2	[1	0	0	1]
3	[0	1	1	0]

Danh sách kề (Adjacency List)

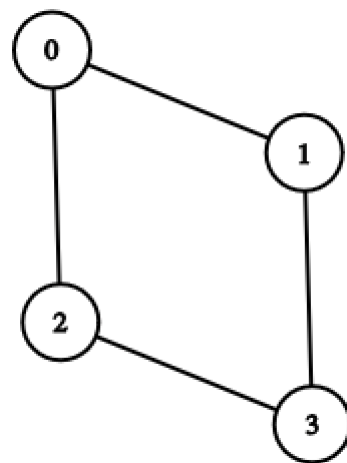
Định nghĩa: Mỗi đỉnh lưu danh sách các đỉnh kề với nó

Ưu điểm:

- Tiết kiệm không gian: $O(V + E)$
- Duyệt hàng xóm nhanh: $O(\text{degree}(v))$
- Phù hợp với đồ thị thưa (sparse graph)

Nhược điểm:

- Kiểm tra cạnh (i,j) tốn $O(\text{degree}(i))$



Danh sách kề:

0: [1, 2]

1: [0, 3]

2: [0, 3]

3: [1, 2]

Biểu diễn đồ thị bằng dictionary

Cách 1: Dictionary với list

Giải thích:

- Key: Đỉnh
- Value: List các đỉnh kề
- Dễ đọc, dễ sử dụng trong Python

```
# Đồ thị vô hướng
graph = {
    'A': ['B', 'C'],
    'B': ['A', 'D'],
    'C': ['A', 'D'],
    'D': ['B', 'C']
}
```

```
# Đồ thị có hướng
directed_graph = {
    'A': ['B'],
    'B': ['D'],
    'C': ['A'],
    'D': ['C']
}
```

Biểu diễn đồ thị có trọng số

Cách 2: Dictionary với list of tuples

Giải thích:

- Mỗi phần tử là tuple (đỉnh_kề, trọng_số)
- ('B', 5) nghĩa là: có cạnh đến B với trọng số 5

```
# Đồ thị có trọng số
weighted_graph = {
    'A': [('B', 5), ('C', 2)],
    'B': [('A', 5), ('D', 4)],
    'C': [('A', 2), ('D', 6)],
    'D': [('B', 4), ('C', 6)]
}
```

Xây dựng đồ thị từ danh sách cạnh

```
edges = [  
    ('A', 'B'),  
    ('A', 'C'),  
    ('B', 'D'),  
    ('C', 'D')  
]
```

```
def build_graph(edges, directed=False):  
    """  
    Xây dựng đồ thị từ danh sách cạnh  
    """  
    graph = {}  
  
    for u, v in edges:  
        # Thêm đỉnh u nếu chưa có  
        if u not in graph:  
            graph[u] = []  
        # Thêm đỉnh v nếu chưa có  
        if v not in graph:  
            graph[v] = []  
  
        # Thêm cạnh u -> v  
        graph[u].append(v)  
  
        # Nếu vô hướng, thêm cạnh v -> u  
        if not directed:  
            graph[v].append(u)  
  
    return graph
```

BFS - Breadth-First Search (Duyệt theo chiều rộng)

Định nghĩa: BFS là thuật toán duyệt đồ thị theo từng lớp (level by level), bắt đầu từ một đỉnh gốc.

- Bắt đầu từ đỉnh gốc
- Thăm tất cả đỉnh kề với gốc (lớp 1)
- Thăm tất cả đỉnh kề với lớp 1 (lớp 2)
- Tiếp tục cho đến khi duyệt hết

Cấu trúc dữ liệu: Sử dụng Queue (hàng đợi - FIFO)

Minh họa BFS

Đồ thị

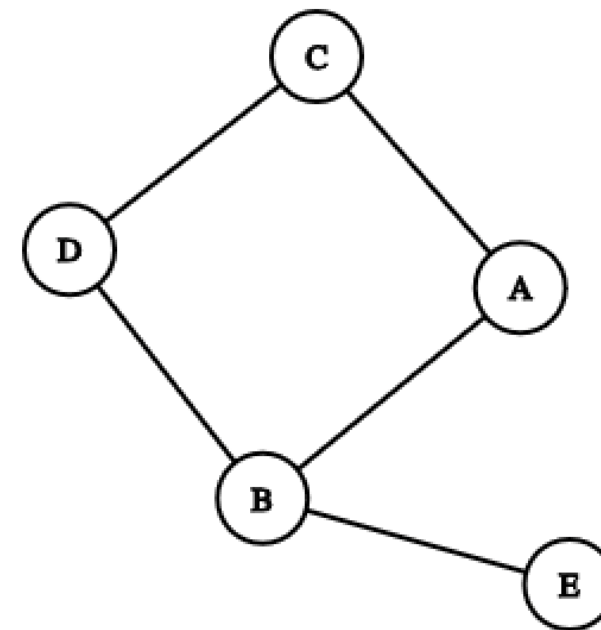
Bắt đầu từ A:

Bước 1: Thăm A, thêm hàng xóm {B, C} vào queue

- Visited: [A]
- Queue: [B, C]

Bước 2: Lấy B từ queue, thăm B, thêm hàng xóm {E, D} vào queue

- Visited: [A, B]
- Queue: [C, E, D]



Bước 3: Lấy C từ queue, thăm C, không thêm gì (đã visited)

- Visited: [A, B, C]
- Queue: [E, D]

Bước 4: Lấy E từ queue, thăm E

- Visited: [A, B, C, E]
- Queue: [D]

Kết quả: $A \rightarrow B \rightarrow C \rightarrow E \rightarrow D$

Bước 5: Lấy D từ queue, thăm D

- Visited: [A, B, C, E, D]
- Queue: []

Thuật toán BFS

```
BFS(graph, start):
```

1. Tạo queue, thêm start vào queue
2. Tạo visited set, đánh dấu start
3. While queue không rỗng:
 - a. Lấy vertex từ queue (dequeue)
 - b. Xử lý vertex (in ra, lưu kết quả...)
 - c. Với mỗi neighbor của vertex:
 - Nếu neighbor chưa visited:
 - + Đánh dấu visited
 - + Thêm vào queue

- Dùng Queue (FIFO - First In First Out)
- Dùng Set để đánh dấu visited (tránh thăm lại)

Code BFS trong Python

```
# Test
graph = {
    'A': ['B', 'C'],
    'B': ['A', 'D', 'E'],
    'C': ['A', 'D'],
    'D': ['B', 'C'],
    'E': ['B']
}

result = bfs(graph, 'A')
print(f"BFS từ A: {result}")
# Output: ['A', 'B', 'C', 'D', 'E']
```

```
from collections import deque

def bfs(graph, start):
    """
    BFS - Duyệt đồ thị theo chiều rộng
    """
    # Khởi tạo
    visited = set()
    queue = deque([start])
    visited.add(start)
    result = []

    # BFS loop
    while queue:
        # Lấy đỉnh từ queue
        vertex = queue.popleft()
        result.append(vertex)

        # Duyệt các đỉnh kề
        for neighbor in graph[vertex]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)

    return result
```

Độ phức tạp của BFS

Thời gian:

- Mỗi đỉnh được thăm đúng 1 lần: $O(V)$
- Mỗi cạnh được xét đúng 1 lần: $O(E)$
- Tổng: $O(V + E)$

Không gian:

- Queue: tối đa $O(V)$ đỉnh
- Visited set: $O(V)$
- Tổng: $O(V)$

DFS - Depth-First Search (Duyệt theo chiều sâu)

Định nghĩa: DFS là thuật toán duyệt đồ thị theo chiều sâu, đi sâu nhất có thể trước khi quay lui.

- Bắt đầu từ đỉnh gốc
- Đi sâu vào một nhánh cho đến khi không còn đường
- Quay lui (backtrack) và thử nhánh khác

Cấu trúc dữ liệu: Sử dụng Stack (ngăn xếp - LIFO) hoặc Đệ quy

Minh họa DFS

Đồ thị

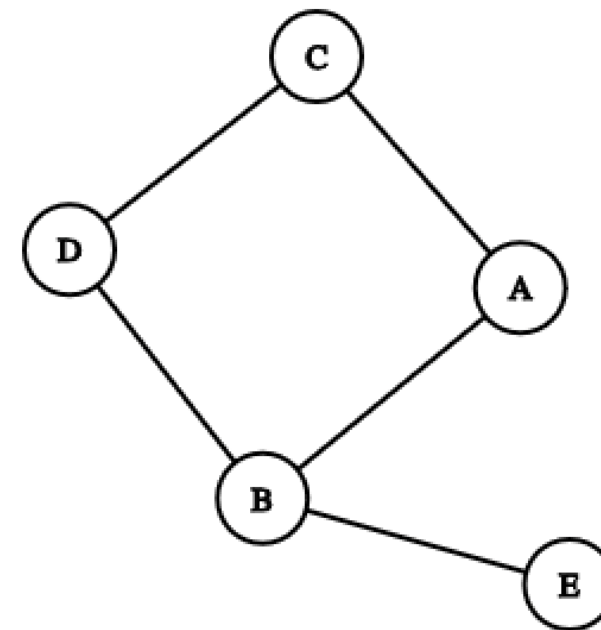
Bắt đầu từ A:

Bước 1: Thăm A, đi sâu vào B (đỉnh kề đầu tiên)

- Visited: [A]
- Stack: [B]

Bước 2: Thăm B, đi sâu vào D (bỏ qua A đã visited)

- Visited: [A, B]
- Stack: [D]



Bước 3: Thăm D, đi sâu vào C (bỏ qua B đã visited)

- Visited: [A, B, D]
- Stack: [C]

Bước 4: Thăm C, không đi sâu được nữa (A, D đã visited)

- Visited: [A, B, D, C]
- Backtrack về D, backtrack về B

Bước 5: Từ B, đi sâu vào E

- Visited: [A, B, D, C, E]

Kết quả: $A \rightarrow B \rightarrow D \rightarrow C \rightarrow E$

Thuật toán DFS

```
DFS_Recursive(graph, vertex, visited):  
    1. Đánh dấu vertex là visited  
    2. Xử lý vertex (in ra, lưu kết quả...)  
    3. Với mỗi neighbor của vertex:  
        - Nếu neighbor chưa visited:  
            + Gọi đệ quy DFS_Recursive(neighbor)
```

Cách 1: Dùng đệ quy (Recursive)

Thuật toán DFS

```
DFS_Iterative(graph, start):
```

1. Tạo stack, thêm start vào stack
2. Tạo visited set
3. While stack không rỗng:
 - a. Lấy vertex từ stack (pop)
 - b. Nếu vertex chưa visited:
 - Đánh dấu visited
 - Xử lý vertex
 - Thêm tất cả neighbors vào stack

Cách 2: Dùng stack (Iterative)

Code DFS đệ quy

```
# Test
graph = {
    'A': ['B', 'C'],
    'B': ['A', 'D', 'E'],
    'C': ['A', 'D'],
    'D': ['B', 'C'],
    'E': ['B']
}

result = dfs_recursive(graph, 'A')
print(f"DFS từ A: {result}")
# Output: ['A', 'B', 'D', 'C', 'E']
```

```
def dfs_recursive(graph, vertex, visited=None, result=None):
    """
    DFS - Duyệt đồ thị theo chiều sâu (đệ quy)
    """
    # Khởi tạo lần đầu
    if visited is None:
        visited = set()
    if result is None:
        result = []

    # Đánh dấu visited
    visited.add(vertex)
    result.append(vertex)

    # Đệ quy với các đỉnh kề
    for neighbor in graph[vertex]:
        if neighbor not in visited:
            dfs_recursive(graph, neighbor, visited, result)

    return result
```

Code DFS dùng stack

```
# Test
result = dfs_iterative(graph, 'A')
print(f"DFS từ A: {result}")
# Output: ['A', 'B', 'D', 'C', 'E']
```

```
def dfs_iterative(graph, start):
    """
    DFS - Duyệt đồ thị theo chiều sâu (dùng stack)
    """
    visited = set()
    stack = [start]
    result = []

    while stack:
        # Lấy đỉnh từ stack
        vertex = stack.pop()

        # Nếu chưa visited
        if vertex not in visited:
            visited.add(vertex)
            result.append(vertex)

            # Thêm các neighbors vào stack (ngược thứ tự)
            for neighbor in reversed(graph[vertex]):
                if neighbor not in visited:
                    stack.append(neighbor)

    return result
```

Độ phức tạp của DFS

Thời gian:

- Mỗi đỉnh được thăm đúng 1 lần: $O(V)$
- Mỗi cạnh được xét đúng 1 lần: $O(E)$
- Tổng: $O(V + E)$

Không gian:

- Stack/Call stack: $O(V)$ trong trường hợp xấu nhất (đồ thị dạng chuỗi)
- Visited set: $O(V)$
- Tổng: $O(V)$

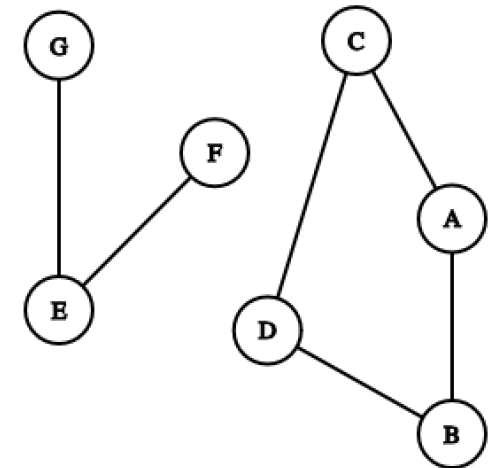
So sánh BFS vs DFS

Tiêu chí	BFS	DFS
Cấu trúc	Queue (FIFO)	Stack (LIFO) hoặc Đệ quy
Thứ tự duyệt	Theo lớp (level by level)	Theo chiều sâu (deep first)
Độ phức tạp	$O(V + E)$	$O(V + E)$
Không gian	$O(V)$	$O(V)$
Tìm đường ngắn	✓ Luôn tìm được	✗ Không đảm bảo
Phát hiện cycle	✓ Có thể	✓ Có thể
Code	Dùng Queue	Đệ quy đơn giản hơn

Connected Components (Thành phần liên thông)

Connected component là một tập con các đỉnh mà:

- Mọi đỉnh trong tập đều kết nối với nhau
- Không có đỉnh nào trong tập kết nối với đỉnh bên ngoài



Có 2 connected components

```
Component 1: {A, B, C, D}  
Component 2: {E, F, G}
```

Tìm Connected Components

- Duyệt tất cả các đỉnh
- Với mỗi đỉnh chưa visited:
 - Chạy BFS/DFS từ đỉnh đó
 - Tất cả đỉnh được thăm tạo thành 1 component
- Đếm số lần chạy BFS/DFS = số components

Code tìm Connected Components

```
# Test với đồ thị có 2 components
graph = {
    'A': ['B', 'C'],
    'B': ['A', 'D'],
    'C': ['A', 'D'],
    'D': ['B', 'C'],
    'E': ['F', 'G'],
    'F': ['E'],
    'G': ['E']
}

count, components = count_connected_components(graph)
print(f"Số components: {count}")
print(f"Chi tiết:")
for i, component in enumerate(components, 1):
    print(f"Component {i}: {component}")

# Output:
# Số components: 2
# Chi tiết:
# Component 1: ['A', 'B', 'C', 'D']
# Component 2: ['E', 'F', 'G']
```

```
def count_connected_components(graph):
    """
    Đếm số connected components trong đồ thị
    """
    visited = set()
    count = 0

    def bfs(start):
        """BFS helper function"""
        queue = deque([start])
        visited.add(start)
        component = []

        while queue:
            vertex = queue.popleft()
            component.append(vertex)

            for neighbor in graph[vertex]:
                if neighbor not in visited:
                    visited.add(neighbor)
                    queue.append(neighbor)

        return component

    # Duyệt tất cả các đỉnh
    components = []
    for vertex in graph:
        if vertex not in visited:
            component = bfs(vertex)
            components.append(component)
            count += 1

    return count, components
```

Ứng dụng của BFS và DFS

BFS:

- Shortest Path (unweighted): Tìm đường đi ngắn nhất
- Level Order Traversal: Duyệt theo lớp (cây, đồ thị)
- Web Crawler: Thu thập dữ liệu web theo breadth
- Social Network: Tìm mức độ kết nối (degrees of separation)

DFS:

- Cycle Detection: Phát hiện chu trình
- Topological Sort: Sắp xếp topo (DAG)
- Path Finding: Tìm đường đi bất kỳ
- Maze Solving: Giải mê cung
- Connected Components: Tìm thành phần liên thông

Cả hai:

- Kiểm tra đồ thị liên thông (connectivity)
- Tìm tất cả đỉnh có thể đến được từ một đỉnh

Tìm đường đi ngắn nhất với BFS

BFS luôn tìm được đường đi ngắn nhất trong đồ thị không có trọng số

Tại sao?

- BFS duyệt theo lớp
- Lớp k chứa các đỉnh cách gốc đúng k bước
- Đỉnh được thăm đầu tiên = đường đi ngắn nhất

Code gợi ý

```
def shortest_path_bfs(graph, start, goal):  
    """  
    Tìm đường đi ngắn nhất từ start đến goal  
    """  
    queue = deque([(start, [start])]) # (vertex, path)  
    visited = {start}  
  
    while queue:  
        vertex, path = queue.popleft()  
  
        if vertex == goal:  
            return path  
  
        for neighbor in graph[vertex]:  
            if neighbor not in visited:  
                visited.add(neighbor)  
                queue.append((neighbor, path + [neighbor]))  
  
    return None # Không tìm thấy đường
```

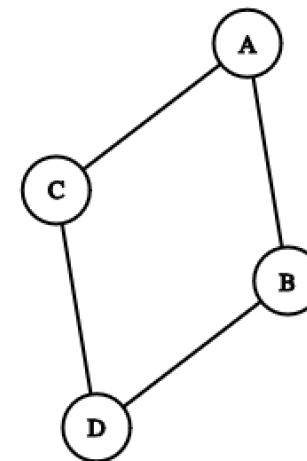
PHẦN 2



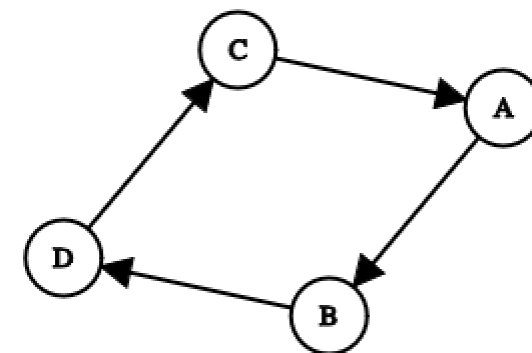
Chu trình (Cycle) là gì?

Định nghĩa: Chu trình là một đường đi bắt đầu và kết thúc tại cùng một đỉnh, đi qua ít nhất 3 đỉnh.

Ví dụ đồ thị vô hướng: Chu trình: $A \rightarrow B \rightarrow D \rightarrow C \rightarrow A$



Ví dụ đồ thị có hướng: Chu trình: $A \rightarrow B \rightarrow D \rightarrow C \rightarrow A$



Tại sao cần phát hiện chu trình?

1. Phát hiện deadlock (bế tắc): Process A chờ B, B chờ C, C chờ A $\rightarrow$ Deadlock!
2. Kiểm tra dependency (phụ thuộc): Package A cần B, B cần C, C cần A $\rightarrow$ Không cài đặt được!
3. Phát hiện circular reference: Object A tham chiếu B, B tham chiếu C, C tham chiếu A $\rightarrow$ Memory leak!

Kết luận: Phát hiện chu trình là bài toán quan trọng trong đồ thị!

Phát hiện chu trình - Đồ thị vô hướng

Trong quá trình DFS, nếu gặp một đỉnh:

- Đã visited VÀ
- Không phải cha của đỉnh hiện tại $\rightarrow$ Có chu trình!

Tại sao cần kiểm tra “không phải cha”?

Vì đồ thị vô hướng: A-B nghĩa là có cả $A \rightarrow B$ và $B \rightarrow A$

- Nếu đang ở A, đi tới B, sau đó B có cạnh về A
- Đây KHÔNG phải chu trình, mà chỉ là đi ngược lại cạnh vừa đi!

Ví dụ phát hiện chu trình vô hướng

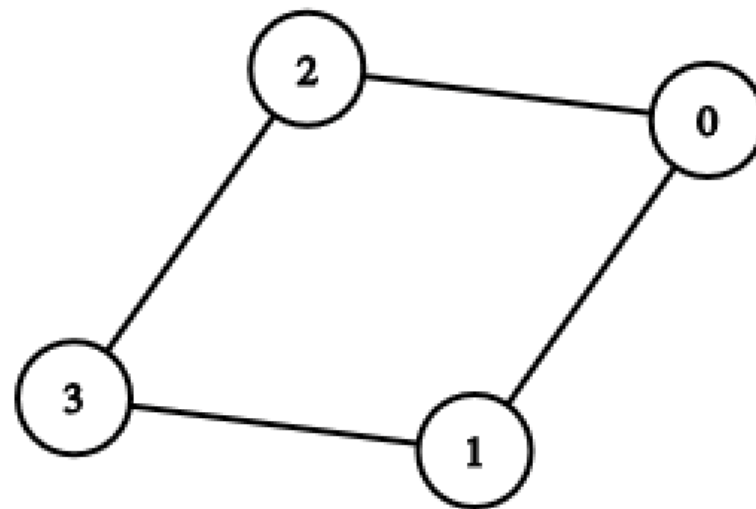
Chạy DFS từ 0:

Bước 1: Thăm 0, đi tới 1 (cha của 1 là 0)

- Visited: {0, 1}
- Path: $0 \rightarrow 1$

Bước 2: Từ 1, đi tới 3 (cha của 3 là 1)

- Visited: {0, 1, 3}
- Path: $0 \rightarrow 1 \rightarrow 3$



Bước 3: Từ 3, đi tới 2 (cha của 2 là 3)

- Visited: {0, 1, 3, 2}
- Path: $0 \rightarrow 1 \rightarrow 3 \rightarrow 2$

Bước 4: Từ 2, gặp 0 (đã visited, không phải cha của 2)

- Phát hiện chu trình: $0 \rightarrow 1 \rightarrow 3 \rightarrow 2 \rightarrow 0$

Thuật toán phát hiện chu trình vô hướng

```
has_cycle_undirected(graph):  
    1. Khởi tạo visited = set()  
    2. Với mỗi đỉnh chưa visited:  
        a. Chạy DFS từ đỉnh đó  
        b. Trong DFS:  
            - Đánh dấu đỉnh hiện tại là visited  
            - Với mỗi neighbor:  
                • Nếu neighbor chưa visited:  
                    → đệ quy DFS(neighbor, current)  
                • Nếu neighbor đã visited VÀ neighbor ≠ parent:  
                    → Có chu trình, return True  
    3. Nếu duyệt hết không tìm thấy → return False
```

Cần truyền tham số parent trong DFS

Kiểm tra neighbor != parent để tránh false positive

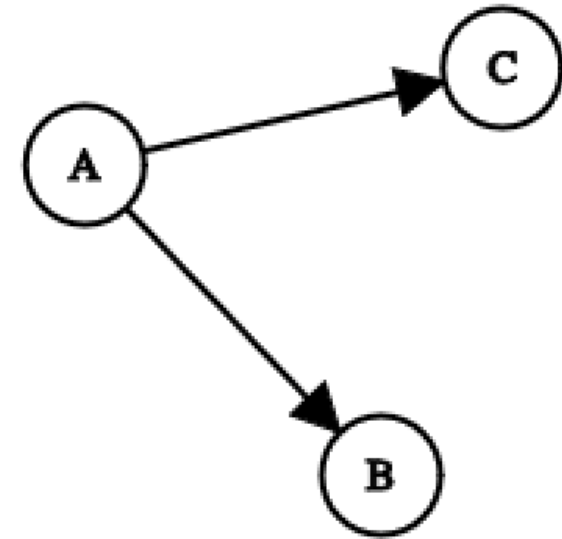
Phát hiện chu trình - Đồ thị có hướng

Đặc điểm đồ thị có hướng:

- Cạnh có chiều: $A \rightarrow B$ (không có nghĩa $B \rightarrow A$)
- Phức tạp hơn đồ thị vô hướng!

Tại sao phức tạp hơn?

- DFS: $A \rightarrow B$, quay lại A, đi $A \rightarrow C$
- Từ C, có thể đi tới B (B đã visited)
- Nhưng KHÔNG có chu trình! ($A \rightarrow B \rightarrow \dots \rightarrow C \rightarrow B$ không thành chu trình vì không quay về A)



Ba trạng thái của đỉnh (Three-Color Approach)

Cần phân biệt 3 trạng thái:

1. WHITE (Trắng) - Chưa thăm:

- Đỉnh chưa được visit
- Giá trị: 0 hoặc không có trong dict

2. GRAY (Xám) - Đang xử lý:

- Đỉnh đang trong recursion stack (call stack)
- Đang DFS từ đỉnh này nhưng chưa hoàn thành
- Giá trị: 1

3. BLACK (Đen) - Đã xử lý xong:

- Đã DFS xong đỉnh này và tất cả con cháu của nó
- Giá trị: 2

Quy tắc phát hiện chu trình:

Nếu trong DFS, gặp một đỉnh GRAY $\rightarrow$ Có chu trình!

Ví dụ ba trạng thái

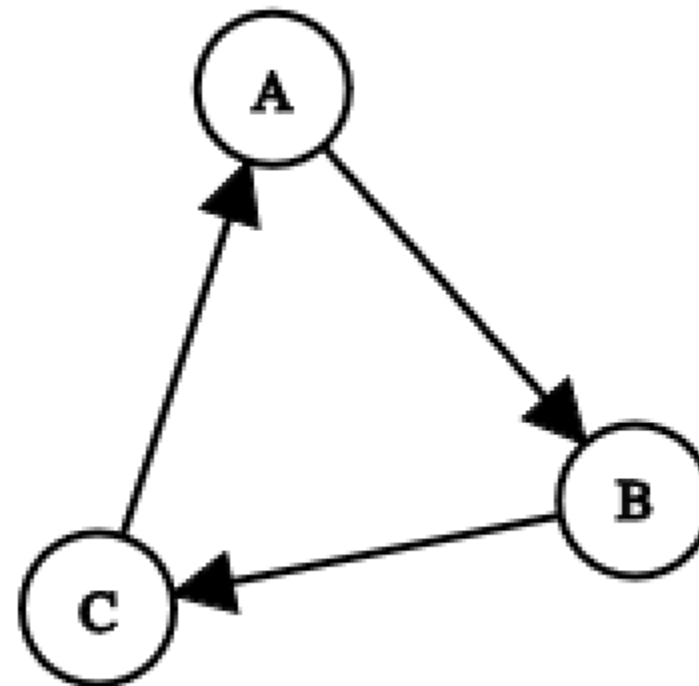
Chạy DFS từ A:

Bước 1: Thăm A, set A = GRAY

- State: {A: GRAY}
- Stack: [A]

Bước 2: Từ A, đi tới B, set B = GRAY

- State: {A: GRAY, B: GRAY}
- Stack: [A, B]



Bước 3: Từ B, đi tới C, set $C = \text{GRAY}$

- State: $\{A: \text{GRAY}, B: \text{GRAY}, C: \text{GRAY}\}$
- Stack: $[A, B, C]$

Bước 4: Từ C, gặp A (A đang GRAY = trong stack)

- Phát hiện chu trình!
- $A \rightarrow B \rightarrow C \rightarrow A$

Thuật toán phát hiện chu trình có hướng

```
has_cycle_directed(graph):  
    1. Khởi tạo color = {} (hoặc dùng WHITE/GRAY/BLACK)  
    2. Với mỗi đỉnh:  
        a. Nếu color[vertex] == WHITE:  
            b. Chạy DFS(vertex)  
    3. Trong DFS(vertex):  
        a. Set color[vertex] = GRAY  
        b. Với mỗi neighbor của vertex:  
            • Nếu color[neighbor] == GRAY:  
                → Có chu trình! Return True  
            • Nếu color[neighbor] == WHITE:  
                → Đệ quy DFS(neighbor)  
                → Nếu tìm thấy chu trình, return True  
        c. Set color[vertex] = BLACK  
        d. Return False  
    4. Nếu duyệt hết không tìm thấy → return False
```


Độ phức tạp phát hiện chu trình

Đồ thị vô hướng:

Thời gian: $O(V + E)$

- Duyệt tất cả đỉnh: $O(V)$
- Duyệt tất cả cạnh: $O(E)$

Không gian: $O(V)$

- Visited set: $O(V)$
- Recursion stack: $O(V)$ trong trường hợp xấu nhất

Đồ thị có hướng:

Thời gian: $O(V + E)$ - giống vô hướng

Không gian: Tổng: $O(V)$ □

- Color dict: $O(V)$
- Recursion stack: $O(V)$

Kết luận: Cả hai đều rất hiệu quả với đồ thị lớn!

So sánh phát hiện chu trình

Tiêu chí	Đồ thị vô hướng	Đồ thị có hướng
Trạng thái	Visited (boolean)	3 màu (WHITE/GRAY/BLACK)
Kiểm tra	neighbor != parent	color == GRAY
Độ phức tạp	$O(V + E)$	$O(V + E)$
Khó	Đơn giản hơn	Phức tạp hơn
Ứng dụng	Network connectivity	Dependency graph

Topological Sort là gì?

Định nghĩa: Topological Sort (Sắp xếp topo) là sắp xếp các đỉnh của đồ thị có hướng sao cho:

- Với mọi cạnh $u \rightarrow v$
- u xuất hiện trước v trong thứ tự

```
Toán cơ bản → Toán nâng cao → Thuật toán
```

```
↓
```

```
Lập trình cơ bản → Cấu trúc dữ liệu → Thuật toán
```

Topological order:

1. Toán cơ bản
2. Lập trình cơ bản
3. Toán nâng cao
4. Cấu trúc dữ liệu
5. Thuật toán

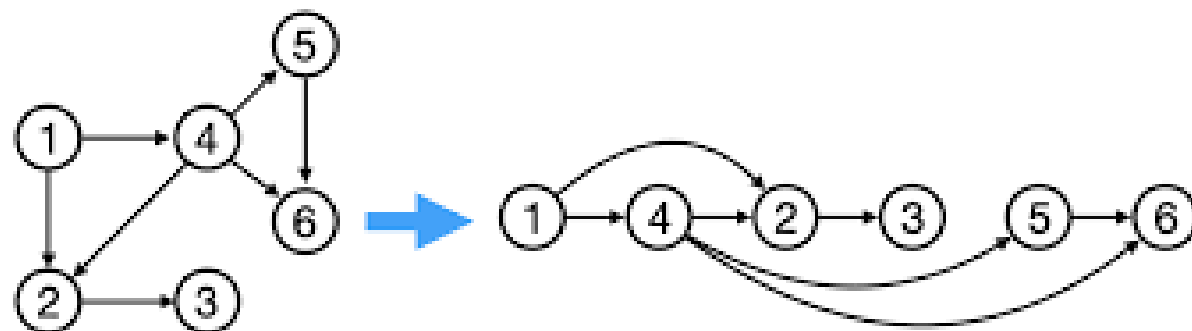
Tại sao cần Topological Sort?

1. Lập lịch công việc (Task Scheduling):

- Task A phải hoàn thành trước Task B
- Tìm thứ tự thực hiện hợp lệ

2. Quản lý dependencies:

- Package A cần B, B cần C
- Thứ tự cài đặt: $C \rightarrow B \rightarrow A$

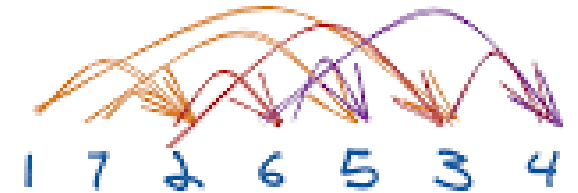
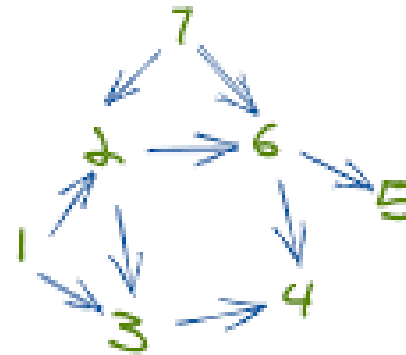


3. Build system:

- File A.cpp include B.h, B.h include C.h
- Thứ tự compile: C.h $\rightarrow$ B.h $\rightarrow$ A.cpp

4. Course Prerequisites:

- Môn A cần học trước môn B
- Sắp xếp thứ tự học hợp lệ



Đặc điểm:

- Có thể có nhiều topological order hợp lệ
- Nếu có chu trình $\rightarrow$ Không tồn tại topological order!

Topological Sort với DFS

Sử dụng DFS và thêm đỉnh vào kết quả khi hoàn thành xử lý (post-order)

Thuật toán:

- Chạy DFS từ mọi đỉnh chưa visited
- Khi hoàn thành xử lý một đỉnh $\rightarrow$ thêm vào stack
- Kết quả = stack đảo ngược (hoặc pop từ stack)

Độ phức tạp: $O(V + E)$ - giống DFS thông thường

Thuật toán Topological Sort

```
topological_sort(graph):  
    1. Kiểm tra đồ thị có chu trình không  
        → Nếu có chu trình → return None (không tồn tại topo sort)  
  
    2. Khởi tạo:  
        - visited = set()  
        - stack = []  
  
    3. Định nghĩa DFS(vertex):  
        a. Đánh dấu vertex là visited  
        b. Với mỗi neighbor của vertex:  
            • Nếu neighbor chưa visited:  
                → Đệ quy DFS(neighbor)  
        c. Thêm vertex vào stack (post-order)  
  
    4. Với mỗi đỉnh chưa visited:  
        a. Gọi DFS(vertex)  
  
    5. Return stack đảo ngược (hoặc pop từ stack)
```




Kết thúc